

Othello

Juan M. Alberola

jalberola@dsic.upv.es

Grado en Tecnologías Interactivas

Recursos

Código:

- <https://gitlab.com/Alberola/othello.git>

Vídeos:

- [01-¿Cómo se juega a Othello?](#)
- [02-Explicación de los scripts](#)
- [03-Script Player](#)

¿Cómo se juega a Othello?

En la parte de **recursos** se proporcionan varios vídeos. Se recomienda leer la práctica y ver cada vídeo en el momento donde se indique.

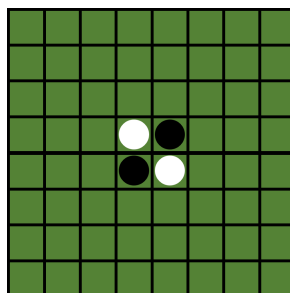


Figura 1: Estado inicial de la partida

Othello es un juego de estrategia de dos jugadores representados por fichas negras y blancas en un tablero de 8 x 8. Inicialmente, se colocan dos fichas de cada jugador en las posiciones que se muestran en la Imagen 1.

El jugador que empieza es el que tiene negras y puede poner una ficha en alguna casilla que esté libre, de manera que se forme una línea vertical, horizontal o diagonal entre la nueva ficha (negra) y una existente (negra), de forma que las fichas que queden entre estas dos sean blancas (1 o más). Por ejemplo, para el escenario inicial, las posibles casillas donde puede moverse negras se muestran en la Imagen 2.

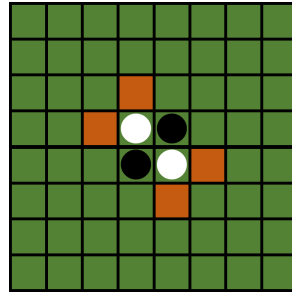


Figura 2: Casillas en las que negras puede poner una ficha

Después de poner una ficha, el jugador de negras voltea todas las fichas blancas que se encuentren atrapadas entre la nueva ficha negra y cualquier ficha negra existente. Todas las fichas blancas volteadas, pasan a ser negras. Por ejemplo, si de las 4 casillas posibles anteriores, negras mueve a la casilla disponible que se encuentra más arriba, se voltearía una blanca y quedaría tal y como se muestra en la Imagen 3.

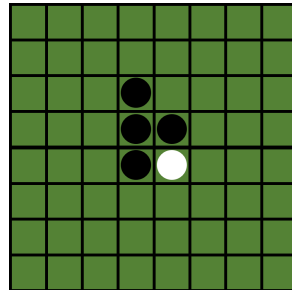


Figura 3: Estado del tablero después de mover negras

Ahora le tocaría el turno a blancas, que realizaría el mismo proceso. Siguiendo el juego anterior, blancas podría mover en las casillas que se muestran en la Imagen 4.

El juego termina cuando hay fichas en las 64 casillas o cuando ningún jugador puede mover. El jugador que gana es el que tiene más fichas en el tablero. Si los dos jugadores tienen el mismo número de fichas, entonces hay un empate.

Puedes ver el vídeo [01-¿Cómo se juega a Othello?](#) donde se muestra una explicación del juego.

Descripción de la práctica a realizar

En esta práctica deberéis implementar en **equipos de 2-3 personas** una IA que juegue contra un humano siguiendo el algoritmo de **MINIMAX**. Para ello, se proporciona un proyecto en **Unity**

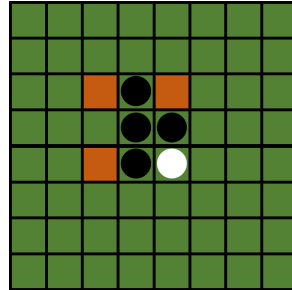


Figura 4: Casillas en las que blancas puede poner una ficha

56	57	58	59	60	61	62	63
48	49	50	51	52	53	54	55
40	41	42	43	44	45	46	47
32	33	34	35	36	37	38	39
24	25	26	27	28	29	30	31
16	17	18	19	20	21	22	23
8	9	10	11	12	13	14	15
0	1	2	3	4	5	6	7

Figura 5: Numeración de las casillas en el proyecto de Unity

con un juego funcional, pero con un movimiento de la máquina aleatorio. Este proyecto lo puedes clonar a partir del repositorio que se indica en la sección de recursos. A partir de este juego, deberás implementar la IA. Si abres el proyecto en Unity, verás los siguientes scripts:

- **Constants:** define algunas constantes del juego.
- **Tile:** representa una casilla del juego. **numTile** es el número de casilla entre 0 y 63 siguiendo el orden que se muestra en la Imagen 5. También tiene un valor de fila y columna por si queremos utilizarlo.
- **TileBehaviour:** captura un clic en una casilla y tiene un objeto de tipo **Tile** que contiene la información de esta casilla. Hay 64 GameObjects que representan las 64 casillas y cada uno tiene este script.
- **Controller:** se encarga de controlar las dinámicas del juego.
- **BoardManager:** es una clase que ofrece varias funciones para operar con tableros.
- **Player:** define el movimiento de la máquina. **Este es el único script que deberás modificar.**

Puedes ver el vídeo [02-Explicación de los scripts](#) donde se explican algunos detalles de cada script.

El script **Player** tiene una función **SelectTile** que es llamada desde el **Controller** cada vez que le toca el turno a la máquina. Esta función recibe el tablero (estado actual) y devuelve la casilla a donde se realiza el movimiento.

Actualmente, esta función devuelve una posición aleatoria de entre las posibles. El objetivo es que implementes el algoritmo **MINIMAX** para elegir el mejor movimiento posible. Para ello, se define la clase de tipo **Node** que representa cada uno de los nodos del árbol **MINIMAX**. Cada nodo tiene los siguientes atributos:

- **board**: define el estado del tablero de este nodo.
- **parent**: define el nodo padre en el árbol **MINIMAX**.
- **childList**: define la lista de nodos hijos en el árbol **MINIMAX**.
- **type**: define el tipo del nodo (**MIN** o **MAX**).
- **utility**: define la utilidad del nodo.
- **alfa** y **beta**: define el valor α o β de este nodo. Sólo se usan en la **parte avanzada** de la práctica.

Cada nodo del árbol tiene un atributo **parent** que apunta a su nodo padre **MINIMAX**. Al mismo tiempo, cada nodo también tiene un atributo **childList** que es una lista de nodos hijos. Cabe destacar que el nodo raíz no tiene padre, mientras que los nodos terminales no tendrán hijos. En la Imagen 6 puedes ver un esquema de un árbol con un nivel.

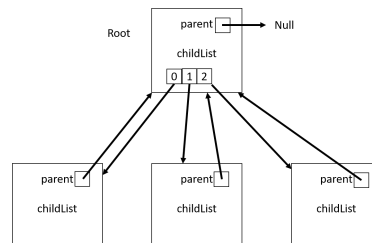


Figura 6: Nodos y atributos parent y childList

La creación del árbol **MINIMAX** para calcular el mejor movimiento a partir del estado actual de la partida, empieza generando un nodo raíz a partir del tablero recibido. Este nodo será de tipo **MAX**. A continuación, se calculan todas las posiciones donde puede mover el jugador y para cada una de ellas, se genera un nuevo nodo hijo, que será de tipo **MIN**. En la Imagen 7 puedes ver un ejemplo de un nodo raíz con tres hijos para cada uno de sus tres movimientos posibles.

Como puedes ver, ya está implementado la generación del primer nivel de nodos del árbol **MINIMAX**. En esta práctica, deberás implementar un **nivel mínimo de profundidad 2** y un **nivel máximo de profundidad 4**. Puedes ver el vídeo [03-Script Player](#) donde se explican los detalles del script **Player**.

Para implementar el árbol **MINIMAX**, deberás seguir los pasos que se indican a continuación:

1. Generar el árbol **MINIMAX** de una profundidad entre 2 y 4.

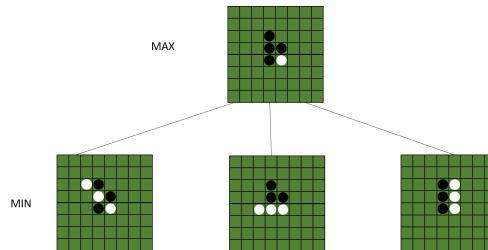


Figura 7: Nodo raíz y tres hijos para un movimiento de blancas

2. Aplicar una función de utilidad a cada nodo terminal (siendo un nodo terminal aquel que esté en tu nivel máximo de profundidad).
3. Usar los valores de utilidad de los nodos terminales para determinar la utilidad de los nodos predecesores. Es un proceso de volcado de valores hasta la raíz por niveles
 - En los nodos **MAX**, se vuelca el mayor valor de sus hijos.
 - En los nodos **MIN**, se vuelca el menor valor de sus hijos.
4. Cuando el nodo raíz tenga ya un valor, elegir el mejor movimiento y utilizarlo como valor de retorno de la función **SelectTile** en vez del valor aleatorio que tiene ahora.

Para realizar estos pasos puedes realizar las modificaciones que quieras en el script **Player**, pero **no toques** el resto de scripts.

Para realizar estos pasos, deberás pensar en algunas cosas, como estas:

- Cómo **recorrer** el árbol desde la raíz a los nodos terminales.
- Cómo **volcar** en un nodo padre el mejor valor de sus nodos hijos.
- Cómo **calcular** la utilidad de cada nodo. Esta parte es **clave**, porque determinará lo buena que es la **IA** que has implementado. Soy consciente que habláis entre vosotros, pero como vamos a hacer una competición, se recomienda que no os deis pistas de cómo calculáis la utilidad de un nodo.

Te puedes apoyar en la clase **BoardManager** a través de la variable **boardManager** que tienes definida en la clase **Player**. A continuación se resumen algunas funciones interesantes que ofrece esta clase:

- **CountPieces**: dado un tablero y un jugador (blancas o negras), devuelve cuántas fichas hay de ese color en el tablero.
- **FindSelectableTiles**: dado un tablero y un jugador (blancas o negras), devuelve un listado con las posiciones donde puede moverse el jugador.
- **FindSwappablePieces**: dado un tablero, una posición y un jugador (blancas o negras), devuelve un listado con las posiciones del oponente que cambiarían de color si el jugador mueve a esta posición.

- **Move**: dado un tablero, una posición y un jugador (blancas o negras), realiza el movimiento del jugador a la posición indicada, modificando el tablero que se pasa por parámetro.
- **PrintBoard**: dado un tablero, lo imprime por pantalla (un 1 es negras, un -1 es blancas y un 0 es posición vacía).

Nota 1: Es posible que uno de los movimientos nos lleve a un tablero donde el jugador contrario no pueda mover, en este caso, pasaría el turno. Esto puede representarse en el árbol MINIMAX como un nodo cuyo atributo **board** es el mismo que el de su padre. Es decir, no haría falta hacer un **boardManager.Move** porque no habría ninguna casilla seleccionable, pero sí que debería generarse un nodo hijo.

Nota 2: La variable **turn** de **Player.cs** controla el turno de cada jugador. Si no la tocas, puedes comprobar que esta variable se pone a -1 para Player2 en la interfaz de Unity. Esto significa, que el Player2 hace movimiento de blancas. Originalmente, este variable no se toca, por lo que en cada turno, le tocará mover siempre las blancas (lógico). Es posible modificar esta variable durante la generación de los nodos, pero es importante que mantenga su valor original cada vez que desde **Controller** se llame a **SelectTile**. Es decir, si modificas **turn** para la generación, **asegúrate** que antes de hacer el **return**, la vuelves a poner a su valor original (-1 en el caso de Player2). Si generamos un número par de nodos (2 o 4), no habrá problema, pero si generamos 3, no funcionará bien si no hemos puesto su valor original.

Nota 3: Si el player AI hace un **return** de una casilla que **no es válida**, pasa el control al humano, con lo que parece que no esté jugando la IA. Por ello, se sugiere poner un **print** por la consola de la casilla elegida para ver a cuál se va a mover y controlar que es buena. Después, ya se podrá quitar este **print**.

Consejos para desarrollar el Minimax:

1. A partir de un nodo raíz, crear un nivel de hijos
2. Comprueba que los hijos que se generan son correctos (con las funciones **FindSelectableTiles** y **PrintBoard**)
3. Crear un segundo nivel de hijos
4. Comprueba que los hijos son correctos
5. Calcular la utilidad de los hijos del 2º nivel (podemos empezar por una simple)
6. Implementar MINIMAX (sin poda)
7. Comprueba que en el nodo raíz se toma la decisión correcta
8. Esta decisión es la que devuelve la función

Diseño de heurísticas y función de utilidad

Una parte fundamental de la práctica consiste en el diseño de la función de utilidad, ya que esta determina el comportamiento de la IA. En lugar de implementar una única función de utilidad, deberéis diseñar y comparar una **familia de heurísticas**, con el objetivo de analizar cómo diferentes criterios de decisión afectan al rendimiento del agente.

Requisito obligatorio: al menos 3 heurísticas

Cada equipo deberá implementar y utilizar al menos **tres heurísticas distintas**, que serán empleadas por el algoritmo de decisión del agente. Estas heurísticas deberán ser claramente diferentes entre sí, evitando variaciones triviales de una misma fórmula o cambios simples de pesos.

- **H1 (heurística básica):** basada en criterios simples, como por ejemplo la diferencia de fichas entre jugadores.
- **H2 (heurística estratégica):** basada en aspectos del juego como la movilidad (número de movimientos disponibles), control del tablero o estabilidad de posiciones.
- **H3 (heurística avanzada):** combinación de varios criterios o diseño propio más elaborado, incorporando una estrategia más compleja. La heurística H3 **no puede ser una simple combinación lineal de H1 y H2** (por ejemplo, del tipo $H3 = a \cdot H1 + b \cdot H2$).

IMPORTANTE: Las heurísticas deben ser realmente diferentes entre sí. No se aceptarán variaciones triviales basadas únicamente en modificar pesos de una misma fórmula.

Opcional: Heurística avanzada

Para llegar a la máxima nota, se requerirá implementar y evaluar una cuarta heurística que incorpore una estrategia no lineal o dependiente de la fase del juego (inicio, medio o final de la partida), adaptando su comportamiento según el estado del tablero. Por ejemplo:

- mayor importancia a la movilidad en fases iniciales
- mayor importancia al control del tablero en fases intermedias
- mayor importancia a la estabilidad o número de piezas en fases finales

Esta condición tiene como objetivo fomentar el diseño de estrategias adaptativas y evitar funciones de evaluación estáticas y simplistas.

Comparativa experimental de heurísticas

Una vez implementadas las distintas heurísticas, cada equipo deberá realizar un estudio experimental para analizar su comportamiento y rendimiento. Para ello, se deberán ejecutar, como mínimo, los siguientes enfrentamientos:

- 2 partidas jugando como jugador **blanco**.
- 2 partidas jugando como jugador **negro**.
- Todas las partidas deberán realizarse contra el mismo oponente de referencia (por ejemplo, un agente aleatorio u otro baseline).

El objetivo es asegurar una comparación consistente y evitar sesgos debidos a diferencias en el oponente.

Los resultados obtenidos deberán presentarse en forma de tabla, incluyendo al menos la siguiente información:

Heurística	Victorias	Fichas medias
H1		
H2		
H3		

Además, se deberán interpretar los resultados. No se espera una descripción larga elaborada por la IA, ya que **es preferible un análisis propio aunque sea más corto**. Se deberá explicar y justificar, por ejemplo:

- Por qué una heurística obtiene mejores resultados que otra.
- En qué situaciones una heurística es más efectiva que otra.
- Qué características del juego favorecen o perjudican a cada heurística.

IMPORTANTE: El valor principal de esta sección no es el resultado final de las partidas, sino la capacidad de análisis y justificación de las diferencias entre heurísticas, y que este análisis no lo haya hecho una IA.

Opcional: Ablation study sobre la heurística avanzada (H3)

Como parte opcional avanzada de la práctica, se deberá realizar un análisis de **ablation study** sobre la heurística más compleja (H3). El objetivo de este análisis es comprender qué componentes de la heurística contribuyen realmente al rendimiento del agente.

La heurística H3 **no puede ser una simple combinación lineal de H1 y H2** (por ejemplo, del tipo $H3 = a \cdot H1 + b \cdot H2$). Esta restricción es necesaria para que el ablation study tenga sentido, ya que de lo contrario se limitaría a reproducir el comportamiento de H1 y H2 sin aportar información adicional.

Para ello, se deberá analizar el impacto de los distintos componentes de H3 eliminándolos o modificándolos de forma controlada. Por ejemplo:

- H3 completa.
- H3 sin un componente relevante (por ejemplo, movilidad o control de esquinas).
- H3 sin transformaciones o condiciones específicas.

El objetivo no es únicamente obtener mejores o peores resultados, sino justificar:

- Qué componentes aportan mayor valor estratégico.
- Qué partes de la heurística son redundantes o poco relevantes.
- Cómo afectan las distintas decisiones de diseño al comportamiento del agente.

IMPORTANTE: Este análisis será evaluado en función de la calidad de la justificación y no únicamente de los resultados obtenidos. Además, al igual que antes, es importante que no lo realice una IA, sino que sea algo propio aunque más simple.

Opcional: Poda alfa-beta

Para llegar a la máxima nota se contempla implementar una poda **alfa-beta**. Estos son los pasos del algoritmo **MINIMAX** con poda **alfa-beta**:

- Cada nodo MAX tiene un valor inicial de $\alpha = -\infty$ y cada nodo MIN un valor $\beta = \infty$.
- Generar un nodo terminal en profundidad y calcular su valor de utilidad.
- Volcar el valor al nodo padre y mantener el mejor valor para el padre (máximo valor para MAX y mínimo para MIN).
- Pregunta de corte sobre el nodo al que hemos volcado:
 - Si es nodo MIN y tiene un valor β menor o igual que el valor α de cualquier nodo MAX predecesor: corte.
 - Si es nodo MAX y tiene un valor α mayor o igual que el valor β de cualquier nodo MIN predecesor: corte.
 - Si hay corte, vamos a 3. Si no hay corte, vamos a 5
- Examinamos el valor que hemos volcado en el paso 3:
 - Si quedan más ramas por explorar, es un valor provisional: vamos al paso 2.
 - Si quedan más ramas por explorar, es un valor definitivo: vamos al paso 3.
 - Si ya no quedan más ramas por explorar, y estamos en el nodo raíz: fin del algoritmo.

Entrega

Se realizará la entrega del proyecto en la plataforma dentro de la fecha límite. Para esta entrega, se tendrá que subir lo siguiente:

- El proyecto de Unity en formato zip.
- Tu script de Player llamado como **PlayerNombreApellido**. Aunque esté dentro del proyecto, se entregará una copia por separado. Asegúrate de no dejar mensajes por pantalla ni de haber modificado otros scripts.
- Un vídeo donde se expliquen y demuestren los aspectos más importantes de la práctica, siguiendo estrictamente el siguiente orden:
 1. **Descripción general del agente implementado:** breve explicación del enfoque general seguido para el código del Minimax y la llamada a la función de utilidad.
 2. **(Opcional) Poda alfa-beta:** si se ha implementado la poda, deberá explicarse también esta parte.
 3. **Explicación del código principal:** descripción de las partes más relevantes de la implementación (sin necesidad de recorrer todo el código línea por línea, pero sí explicando la lógica general del agente).
 4. **Explicación de las heurísticas implementadas:**

- H1 (heurística básica).
- H2 (heurística estratégica).
- H3 (heurística avanzada).

Se deberá explicar claramente en qué consiste cada una y qué criterio de decisión utiliza.

5. **Demostración en ejecución:** ejecución del programa mostrando partidas reales utilizando las tres heurísticas, evidenciando que producen comportamientos diferentes.
 6. **(Opcional) Heurística H4:** en caso de haber implementado la heurística dependiente de la fase del juego, se deberá explicar su funcionamiento y mostrar su comportamiento en ejecución.
- En el vídeo indica también si has hecho uso de herramientas como Copilot o Chatgpt y para qué lo has utilizado. En caso de no haber utilizado nada, se mencionará explícitamente.

Criterios de evaluación

- **Implementación básica del agente (6 puntos):** implementación correcta del algoritmo Minimax junto con las **tres heurísticas obligatorias (H1, H2 y H3)** y la realización de la comparativa experimental correctamente justificada (tabla de resultados y análisis). Se valorará especialmente la coherencia entre las heurísticas implementadas, los resultados obtenidos y las conclusiones del análisis, así como que no se detecte el uso de la IA en el análisis.
- **Poda alfa-beta (1 punto):** implementación correcta de la optimización mediante poda alfa-beta sobre el algoritmo de búsqueda.
- **Ablation study sobre H3 (1 punto):** análisis de ablation correctamente justificado, evaluando el impacto de los distintos componentes de la heurística avanzada. Se valorará que no se detecte el uso de la IA en el análisis.
- **Heurística adicional H4 (1 punto):** implementación de una cuarta heurística dependiente de la fase del juego (inicio, medio o final), con comportamiento adaptativo y comparación con el resto de heurísticas.
- **Vídeo explicativo (1 punto):** entrega de un vídeo que cumpla correctamente con todos los requisitos especificados en el enunciado, incluyendo explicación del sistema, heurísticas, demostración en ejecución, comparativa experimental y conclusiones.
- Cualquier modificación de estos criterios, se informará con suficiente antelación